

Estructuras de control

Programación I

UNRN

Universidad Nacional
de Río Negro

06

Agost

0

Programación I

Clase 02: Estructuras de control (+acumuladores) en C

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

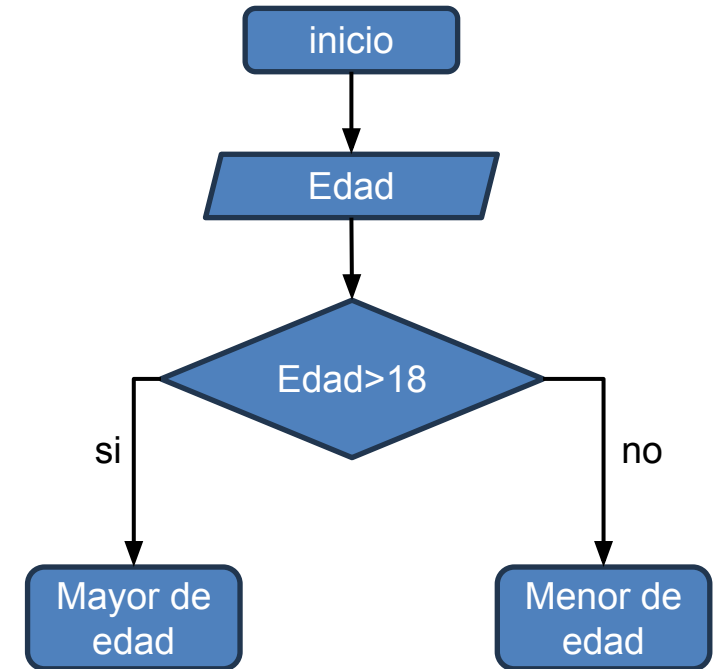
Prof. Daniel Teira/Diego Diaz

Sede Bariloche

1. Condicionales

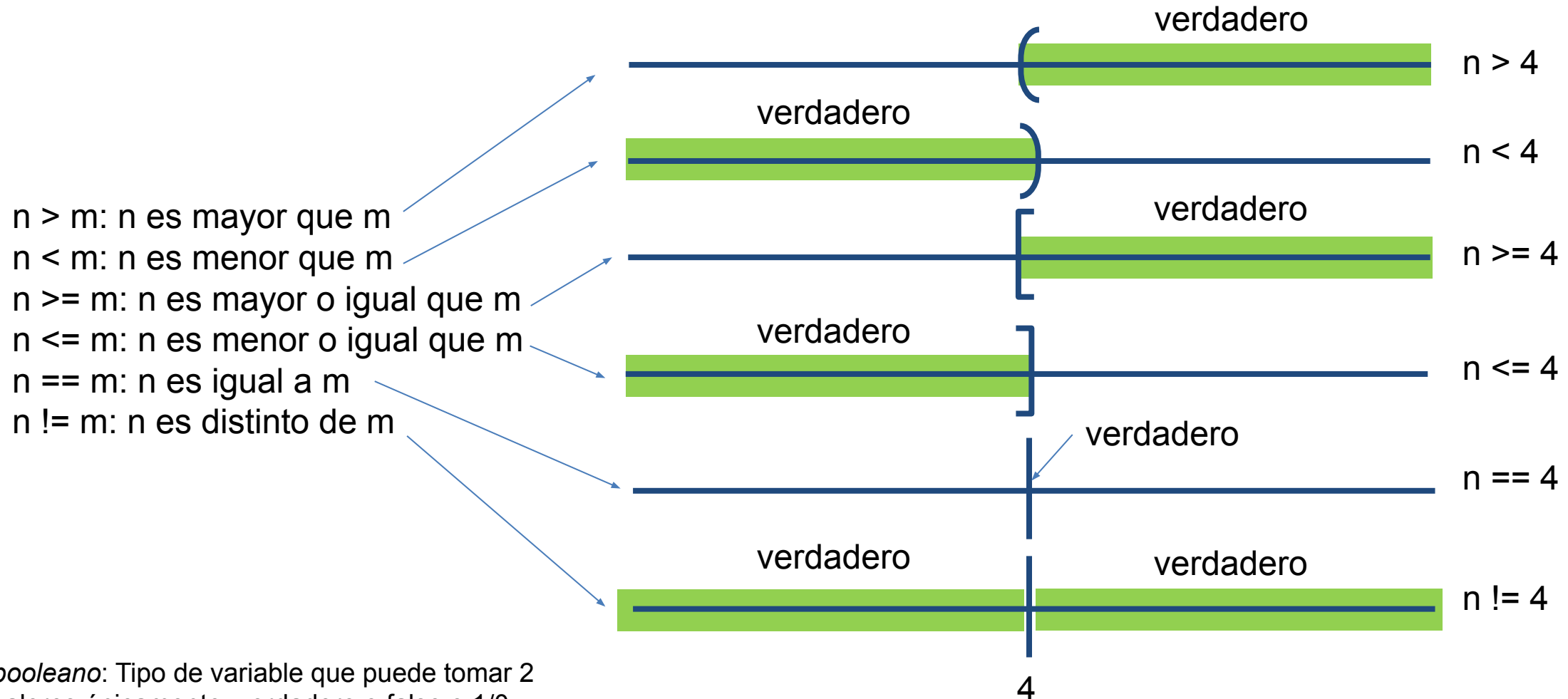
*“Un **condicional** es una instrucción que permite al programa tomar decisiones y elegir qué acciones ejecutar basándose en el cumplimiento de una condición”*

- La Sentencia “if”, “if-else”, Anidamiento y Escaleras “else-if”.
- Bucles e Iteraciones: “while”, “do-while” y “for”.
- Estructura Switch: Casos Discretos, La Sentencia break.
- Contadores vs Acumulador



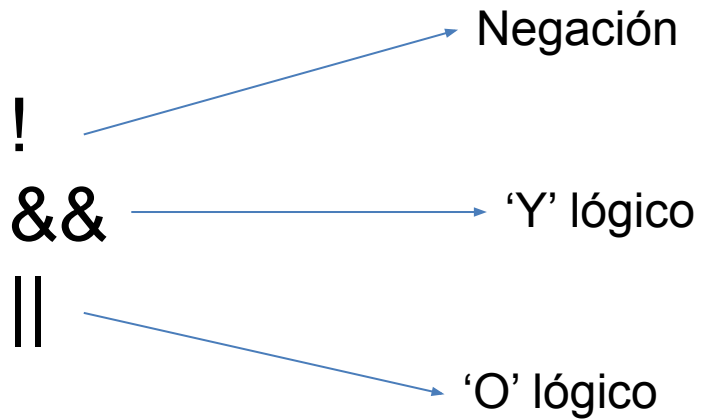
2. Intro: Operadores relacionales

comparan dos valores y devuelven un resultado lógico (*booleano**): 1 si la condición es verdadera y 0 si es falsa, donde n y m , son variables numéricas o expresiones que representan valores cuantificables

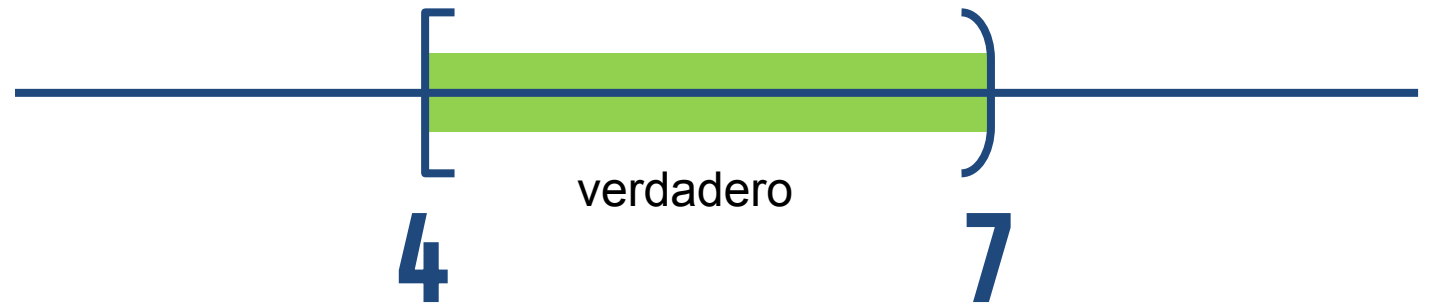


booleano: Tipo de variable que puede tomar 2 valores únicamente, verdadero o falso o 1/0

2. Intro: Operadores lógicos (*para condiciones más complejas*)

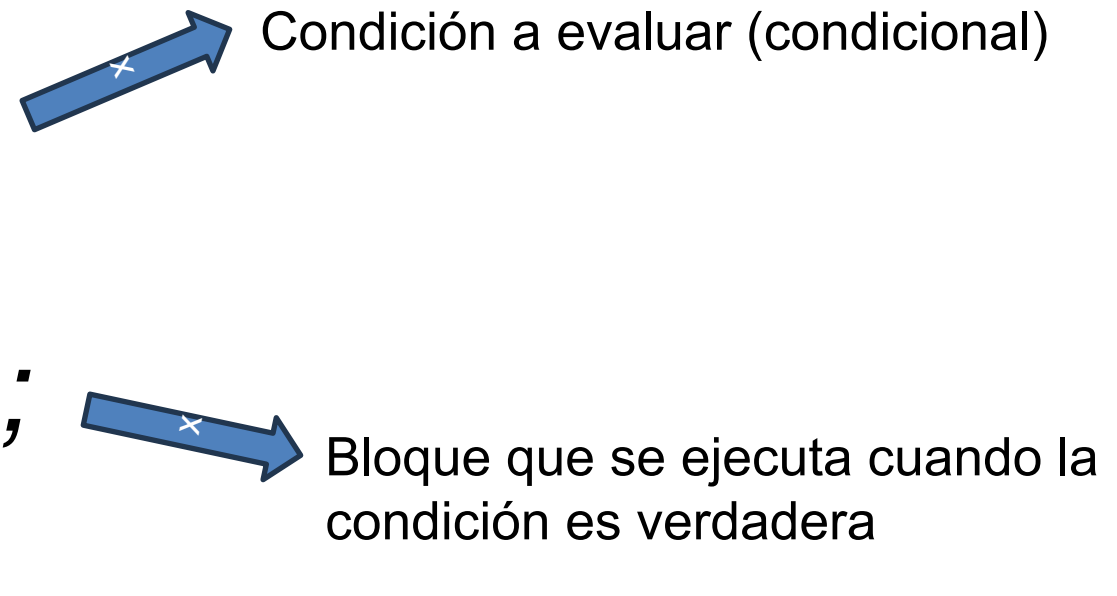


(nota ≥ 4) && (nota < 7)



3. "if" Condicional Simple:

```
if (valor_booleano)
{
    bloque verdadero;
}
```

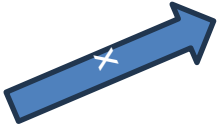


Condición a evaluar (condicional)

Bloque que se ejecuta cuando la condición es verdadera

3. "if" Condicional Simple : Ejemplo

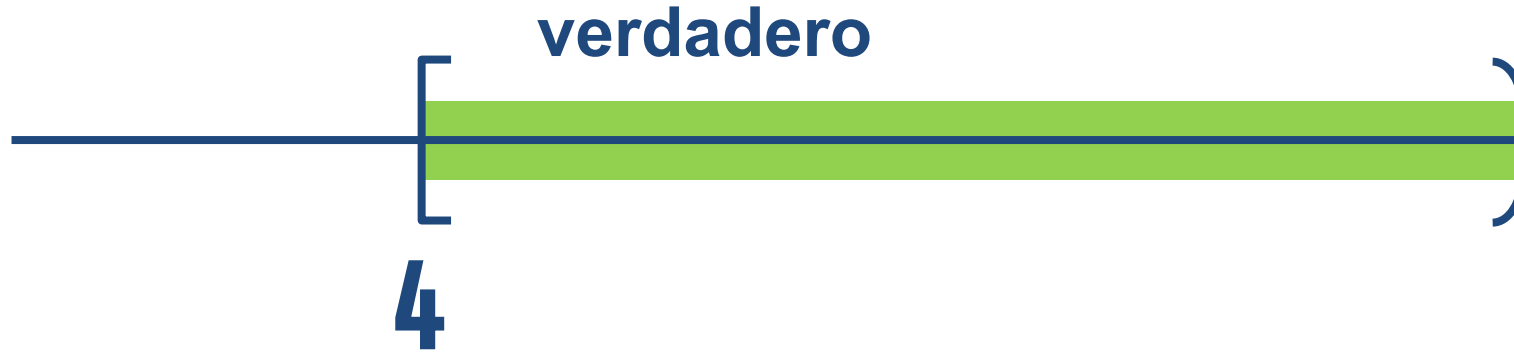
```
if (nota >= 4)
{
    printf("Aprobado\n");
}
```



La condición es mayor o igual a cuatro, en otro caso no se ejecuta lo que está dentro del bloque.

3. "if" Condicional Simple : Ejemplo

(nota ≥ 4)



(nota ≥ 4) && (nota < 7)



3. “if” simple: Ejemplo

```
bool comparar = (nota >= 4) && (nota < 7);  
if (comparar) // (nota >= 4) && (nota < 7);  
{  
    printf("Aprobado\n");  
}
```



3. "if/else": Condicional Doble

```
if (condición)
{
    bloque verdadero;
}
else
{
    bloque falso;
}
```



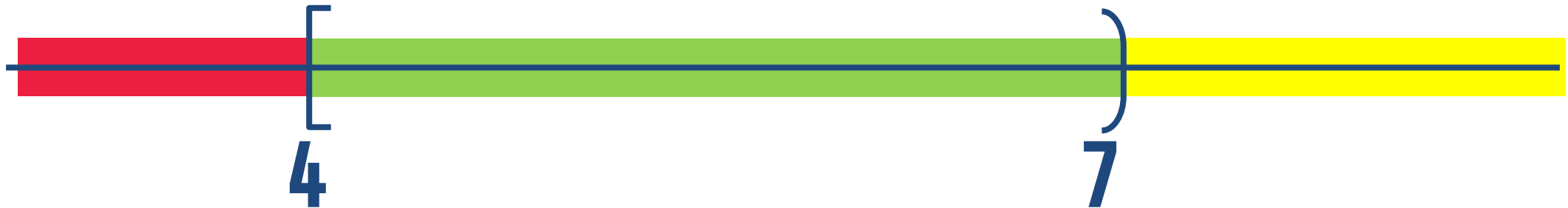
```
if (nota >= 4)
{
    printf("Aprobado\n");
}
else
{
    printf("Desaprobado\n");
}
```



3. “if/else, if/else”: Condicionales Múltiples

```
if (condición)
{
    bloque condición verdadero;
}
else if(otra_condición)
{
    bloque otra_condición verdadero;
}
```

```
if (nota >= 4 && nota < 7)
{
    printf("Aprobado\n");
}
else if(nota < 4)
{
    printf("desaprobado\n");
}
else
{
    printf("promocionado\n");
}
```



3. Resumen de casos de “if, then, else”

1. “if” simple:
2. “if/else”: condiciones excluyentes
3. “if, if/else”, else:

Resumen de reglas para “if, else if, else”

- A. **Estructura limpia:** Todos los bloques van con llaves { } y en su propia línea.
- B. **Exclusividad:** Se ejecuta **solo un** camino (el primero que coincida de arriba hacia abajo).
- C. **Cuidado con el !=** Usar == para comparar. El = es para asignar.
- D. **El ; prohibido:** Jamás poner punto y coma inmediatamente después del paréntesis del `if`.
- E. **Superposiciones:** Cuidado con el orden de los rangos (ej. evaluar `< 10` antes de `< 100`).

3. Bucles e Iteraciones: “for”.

```
for (inicialización; condición; actualización)
{
    bloque; // Código a repetir
}
```

Ejemplos

```
for (int i = 0; i < 10; i++)
{
    printf("%d\n", i);
}
```

```
for (int i = 10; i > 0; i--)
{
    printf("%d\n", i);
}
```

3. Bucles e Iteraciones: “while” y “do-while”

“Primero piensa, luego actúa”

```
while  
(condición)  
{  
    bloque;  
}
```

```
#include <stdio.h>  
  
int main() {  
    int segundos = 3;  
  
    printf("Iniciando cuenta regresiva:\n");  
  
    while (segundos > 0) {  
        printf("%d...\n", segundos);  
        segundos--; // Restamos 1 en cada vuelta  
    }  
  
    printf("lift-off \n");  
    return 0;  
}
```

Iniciando cuenta regresiva:
3...
2...
1...
lift-off

3. Bucles e Iteraciones: “while” y “do-while”

“Primero actúa, luego piensa”

```
do
{
    bloque;
}
while (condición);
```

```
#include <stdio.h>

int main() {
    int bateria = 30; // El teléfono arranca con 30% de carga

    printf("--- Iniciando simulación de uso de batería ---\n");

    do {
        printf("Batería actual: %d%%\n", bateria);

        // El teléfono gasta un 10% de energía por cada tarea
        realizada
        bateria = bateria - 10;

    } while (bateria > 0); // Se repite MIENTRAS la batería sea
    mayor a 0

    printf("Batería agotada (0%%). El teléfono se ha apagado.\n");
    return 0;
}
```

--- Iniciando simulación de uso de
batería ---
Batería actual: 30%
Batería actual: 20%
Batería actual: 10%
Batería agotada (0%). El teléfono se
ha apagado.

3. Lazos mas comunes (ejemplos)

De menor a mayor

```
int numero = 0;
while (numero < 10)
{
    printf("numero:%d", numero);
    numero = numero + 1;
}
```

```
int numero = 0;
do
{
    printf("numero:%d", numero);
    numero = numero + 1;
}
while (numero < 10);
```

De menor a mayor

De mayor a menor

```
int numero = 10;
while (numero > 0)
{
    printf("numero:%d", numero);
    numero = numero - 1;
}
```

```
int numero = 10;
bool bandera = true;
while (bandera)
{
    printf("numero:%d", numero);
    numero = numero - 1;
    if (numero > 0)
    {
        bandera = false;
    }
}
```

De mayor a menor

3. Manipulaciones de lazos: Break

Break: sirve para romper y salir inmediatamente del bucle, sin importar si la condición original todavía se cumple. El programa salta directo a la primera línea de código que esté después del bucle.

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            printf("¡Encontré el 3! Rompemos el bucle con break.\n");
            break; // Sale del for por completo
        }
        printf("Número actual: %d\n", i);
    }
    printf("Ya estoy fuera del bucle.\n");
    return 0;
}
```

Número actual: 1
Número actual: 2
¡Encontré el 3! Rompemos el bucle con break.
Ya estoy fuera del bucle.

3. Manipulaciones de lazos: Continue

Continue no rompe el bucle, sino que se salta el resto del código de la iteración actual y pasa directo a la siguiente vuelta (en un for, va directo al incremento; en un while, vuelve a evaluar la condición).

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        if (i % 2 != 0) {
            // Si el número es impar, saltamos a la siguiente vuelta
            continue;
        }
        printf("Número par encontrado: %d\n", i);
    }
    printf("Bucle terminado.\n");
    return 0;
}
```

Número par encontrado: 2
Número par encontrado: 4
Bucle terminado.

3. Scanf

sirve para escuchar y leer lo que el usuario escribe en el teclado, y luego guardar esa información dentro de una variable.

```
#include <stdio.h>

int main() {
    int edad; // 1. Creamos la "caja" vacía

    printf("¿Cuántos años tienes? ");

    // 2. Usamos scanf para leer el teclado
    scanf("%d", &edad);

    printf("¡Vaya! Tienes %d años.\n", edad);
    return 0;
}
```

¿Cuántos años tienes? 10
¡Vaya! Tienes 10 años.

4. Switch/switch case

El **switch** (o **switch case**) en C es como un menú de opciones o un selector de caminos. Se usa cuando tienes una sola variable y quieres que el programa tome rutas distintas dependiendo de su valor exacto.

En C, solo funciona con números enteros (`int`) o caracteres (`char`).

Son los diferentes "opciones" o escenarios posibles. Si opción vale 1, el programa salta directo al case 1:

¡Esto es vital! El `break` le dice al programa: "Ya terminé con este caso, sal del switch". Si lo olvidas, el programa seguirá ejecutando los casos de abajo en cascada (imprimiría el caso 1, el 2, el 3, etc.).

```
#include <stdio.h>
```

```
int main() {  
    int opción;
```

```
    printf("Elige una opción (1-3): ");  
    scanf("%d", &opción);
```

```
    switch (opción) {  
        case 1:
```

```
            printf("Elegiste la opción 1: Ver perfil.\n");  
            break; // Importante: detiene el switch aquí
```

```
        case 2:
```

```
            printf("Elegiste la opción 2: Configuración.\n");  
            break;
```

```
        case 3:
```

```
            printf("Elegiste la opción 3: Salir del programa.\n");  
            break;
```

```
        default:
```

```
            printf("Opción no valida. Por favor elige 1, 2 o 3.\n");  
            break;
```

```
    }
```

```
    return 0;
```

```
}
```

Es el plan de emergencia. Se ejecuta **solo si** la variable no coincidió con ninguno de los `case` anteriores

5. Resumen de la clase

- **Condicionales**
- **Operadores relacionales**
- **Operadores lógicos**
- **“if” Condicional Simple**
- **“if/else, if/else”: Condicionales Múltiples**
- **Bucles e Iteraciones: “for”**
- **Bucles e Iteraciones: “while” y “do-while”**
- **Manipulaciones de lazos: Break & continue**
- **Scanf**
- **Switch/switch case**